

Epic 5: Application Building

Story 2: Flask Application Development and Model Integration

The AI Flood Prediction System is deployed as a web application using the Flask framework. Flask acts as the bridge between the trained machine learning model and the user interface, enabling users to enter weather parameters and receive real-time flood predictions through a web browser.

Application Initialization

The Flask application begins by importing the required Python libraries, including Flask, Pandas, and Joblib. The saved machine learning model (floods.save) and the fitted StandardScaler (transform.save) are loaded during application startup. These files are used to preprocess user inputs and generate flood predictions without retraining the model.

URL Routing

Flask uses the `@app.route()` decorator to map URLs to different HTML pages stored in the `templates/` folder.

- `/` – Displays the Home Page (home.html).
- `/Predict` – Opens the Prediction Page (index.html), where users enter weather parameters.
- `/chance` – Displays the Flood Alert Page (chance.html) when the model predicts a flood.
- `/no_chance` – Displays the Safe Status Page (no_chance.html) when no flood risk is detected.

Prediction Process

When the user submits the prediction form, the application receives the weather parameters through an HTTP POST request.

The application performs the following steps:

1. Collects the user input from the web form.
2. Converts the input into a Pandas DataFrame containing the required weather features.
3. Applies the saved StandardScaler to standardize the input values.
4. Loads the saved XGBoost model and generates the flood prediction.
5. Redirects the user to either the Flood Alert page or the Safe Status page based on the prediction result.

Application Execution

The Flask application is started using the `app.run()` method, which launches the development server on localhost. Once running, users can access the application through a web browser and obtain real-time flood risk predictions.

Outcome

- Successfully developed the Flask web application.
- Integrated the trained XGBoost model and StandardScaler.
- Implemented URL routing for all application pages.
- Enabled real-time flood prediction using user-provided weather data.
- Created an interactive and user-friendly web interface for flood risk assessment.

Figure 19: Flask application integrating the trained XGBoost model with the web interface for real-time flood prediction.

```
from flask import Flask, render_template, request
# used to run/serve our application
# render_template is used for rendering the html pages
#import load from joblib to load the saved model file
from joblib import load
```

Create Flask app and Load our model file.

```
app=Flask(__name__) # our flask app
#load model file
model =load('floods.save')
sc=load('transform.save')
```